

Java Swing Detailed Explanation (Based on AVL Visualizer Project)

1. Import Statements

Code Snippet:

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
import java.util.*;
```

Explanation: - **javax.swing:** The core package for all Swing components like `JFrame`, `JPanel`, `JButton`, and `JTextArea`. - **java.awt:** Contains classes for creating GUI elements, layouts, colors, and fonts. - **java.awt.event:** Used for handling user events such as button clicks and mouse actions. - **java.util:** Provides utility classes like `ArrayList`, `List`, and `Random` used in logic and data management.

2. JFrame (Main Window)

Code Snippet:

```
public class AVLVisualizer extends JFrame {
    public AVLVisualizer() {
        super("AVL Tree Visualizer");
        setSize(800, 600);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new BorderLayout());
        setVisible(true);
    }
}
```

Explanation: - **JFrame** is the main window of the GUI. - **super()** sets the window title. - **setSize()** defines the window's dimensions. - **setDefaultCloseOperation()** ensures the program terminates when the window closes. - **setLayout()** applies a layout manager for organizing components. - **setVisible(true)** makes the window appear on screen.

3. JPanel (Drawing Area)

Code Snippet:

```

JPanel drawPanel = new JPanel() {
    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        // Draw nodes and connections here
    }
};

```

Explanation: - **JPanel** serves as a container for custom graphics. - The **paintComponent(Graphics g)** method is overridden to draw shapes like circles (for nodes) and lines (for edges). - **Graphics** provides methods like `drawOval()` and `drawLine()` for rendering visuals.

4. JButton (Control Buttons)

Code Snippet:

```

JButton nextButton = new JButton("Next");
JButton prevButton = new JButton("Previous");

```

Explanation: - **JButton** creates clickable buttons. - These buttons are used to navigate between AVL Tree rotations (Next/Previous). - Each button can trigger specific actions using **ActionListener**.

5. JTextArea (Display Area)

Code Snippet:

```

JTextArea explanationArea = new JTextArea();
explanationArea.setEditable(false);
explanationArea.setLineWrap(true);

```

Explanation: - **JTextArea** displays multiline text. - Used here to show explanations for each AVL rotation or action. - **setEditable(false)** ensures users cannot type into it. - **setLineWrap(true)** wraps long lines of text neatly.

6. JScrollPane (Scroll Wrapper)

Code Snippet:

```

JScrollPane scrollPane = new JScrollPane(explanationArea);

```

Explanation: - **JScrollPane** allows scrolling when the text exceeds visible space. - It's wrapped around `JTextArea` to make long explanations readable.

7. Layout Managers

Code Snippet:

```
setLayout(new BorderLayout());  
add(drawPanel, BorderLayout.CENTER);  
add(controlPanel, BorderLayout.SOUTH);
```

Explanation: - **BorderLayout** divides the window into five regions — North, South, East, West, and Center. - Here: - `drawPanel` (AVL visualization) is at the **center**. - `controlPanel` (buttons and explanation area) is at the **bottom** (South).

8. ActionListener (Handling Button Clicks)

Code Snippet:

```
nextButton.addActionListener(e -> showNextRotation());  
prevButton.addActionListener(e -> showPrevRotation());
```

Explanation: - **ActionListener** detects when a button is clicked. - The **lambda expression** (`e -> ...`) defines what happens after the event. - `showNextRotation()` and `showPrevRotation()` are custom methods that update the visualization.

9. SwingUtilities (Thread-Safe UI Updates)

Code Snippet:

```
SwingUtilities.invokeLater(() -> drawPanel.repaint());
```

Explanation: - **SwingUtilities.invokeLater()** ensures GUI updates occur on the **Event Dispatch Thread (EDT)**. - Swing is single-threaded, so this method prevents UI freezing or race conditions. - **repaint()** refreshes the `JPanel`, causing `paintComponent()` to be called again.

10. Custom Painting in Swing

Code Snippet:

```
protected void paintComponent(Graphics g) {
    super.paintComponent(g);
    g.setColor(Color.BLUE);
    g.fillOval(x, y, width, height);
}
```

Explanation: - **paintComponent()** is the heart of all custom rendering in Swing. - **super.paintComponent(g)** clears the previous drawings. - **Graphics** allows shapes and colors to be drawn. - **Color** is a class from `java.awt` that defines custom colors.

11. Event Handling Flow

Code Snippet:

```
nextButton.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        avlTree.nextRotation();
        drawPanel.repaint();
    }
});
```

Explanation: - **ActionEvent** is generated when a button is clicked. - The **ActionListener** object listens for it and triggers appropriate logic. - After the event, the AVL tree changes and the screen updates.

12. Repainting and Animation

Code Snippet:

```
avlTree.addListener(() -> drawPanel.repaint());
```

Explanation: - Each time the AVL tree changes (e.g., after insertion or rotation), the listener calls `repaint()`. - This updates the visual tree instantly. - It simulates animation through continuous re-rendering of updated graphics.

13. Integration of Swing with Logic

Code Snippet:

```
private List<Runnable> listeners = new ArrayList<>();
void notifyChange() {
    SwingUtilities.invokeLater(() -> {
        for (Runnable r : listeners) r.run();
    });
}
```

Explanation: - Combines **logic layer (AVL tree)** and **GUI layer (Swing)**. - Listeners automatically inform the interface about internal tree changes. - This keeps visuals and data synchronized in real-time.

14. Basic Swing Theory Summary

- **Swing** is a GUI toolkit built on top of **AWT**.
 - It is **lightweight**, **platform-independent**, and **component-based**.
 - **Event-driven programming**: Swing reacts to user actions through listeners.
 - **Painting mechanism**: Uses `paintComponent()` and `Graphics` for visuals.
 - **Layout Managers**: Handle component positioning.
 - **Thread Safety**: Swing components must be updated only on the **Event Dispatch Thread (EDT)**.
-

15. How Swing Works in AVL Visualizer

1. The **JFrame** is created → acts as the main window.
 2. The **JPanel** draws the AVL tree graphics.
 3. **JButtons** trigger next/previous actions.
 4. **ActionListeners** respond to user clicks.
 5. **SwingUtilities** ensures safe repainting.
 6. **JTextArea + JScrollPane** display rotation explanations.
 7. The **AVL logic layer** notifies the GUI whenever a change occurs.
-

Final Note: This implementation effectively demonstrates Swing's event-driven and paint-based architecture by connecting user interactions (Next/Prev buttons) with live AVL tree visualization updates.